

LAPORAN PRAKTIKUM STRATEGI ALGORITMA

MATA KULIAH STRATEGI ALGORITMA



DISUSUN OLEH:

NAMA : RADITYA RIZKI RAHARJA
NIM : 714240041
KELAS : 2A

DOSEN PENGAMPU:

RONI HABIBI, S.KOM., M.T., SFPC

PROGRAM STUDI DIV TEKNIK INFORMATIKA
UNIVERSITAS LOGISTIK & BISNIS INTERNASIONAL
BANDUNG

2026

DAFTAR ISI

DAFTAR ISI	i
DAFTAR TABEL	ii
DAFTAR GAMBAR	iii
1 KOMPLEKSITAS ALGORITMA	1
1.1 Kompleksitas Algoritma	1
1.2 Notasi Big-O	1
1.3 Selection Sort	1
1.4 Insertion Sort	1
1.5 Langkah Praktikum: Selection Sort	3
1.6 Post Test: Insertion Sort	5
1.7 Implementasi Nyata	7
2 ALGORITMA BRUTE FORCE	9
2.1 Algoritma Brute Force	9
2.2 Perpangkatan	9
2.3 Pencarian Elemen Terbesar	9
2.4 Perkalian Dua Matriks	9
2.5 Uji Bilangan Prima	10
2.6 Langkah Praktikum	10
2.7 Post Test	14
2.8 Implementasi Nyata	15
3 ALGORITMA GREEDY	17
3.1 Algoritma Greedy	17
3.2 Masalah Penukaran Uang (Coin Exchange)	17
3.3 Langkah Praktikum: Coin Exchange	18
3.4 Post Test	19
3.5 Implementasi Nyata	21
4 ALGORITMA DIVIDE & CONQUER	23
4.1 Algoritma Divide and Conquer	23
4.2 MinMaks Divide and Conquer	23
4.3 Merge Sort	23
4.4 Quick Sort dan Partisi	24
4.5 Langkah Praktikum: Program Sorting	25

4.6	Post Test: Merge Sort Standalone	29
4.7	Implementasi Nyata	31
5	ALGORITMA DECREASE & CONQUER	33
5.1	Algoritma Decrease and Conquer	33
5.2	Selection Sort sebagai Decrease and Conquer	34
5.3	Langkah Praktikum: Selection Sort D&C	35
5.4	Post Test: Selection Sort dengan Input Pengguna	36
5.5	Implementasi Nyata	37
6	ALGORITMA BFS DAN DFS	39
6.1	Traversal Graf	39
6.2	Breadth First Search (BFS)	39
6.3	Depth First Search (DFS)	39
6.4	Langkah Praktikum: BFS dan DFS pada Graf 6.1	42
6.5	Post Test: BFS dan DFS pada Graf Pre Test (Gambar 6.4)	46
6.6	Implementasi Nyata	48

DAFTAR TABEL

1	Operasi Perbandingan Per Pass Selection Sort	4
2	Perbandingan Selection Sort dan Insertion Sort	7
3	Kompleksitas Operasi Uji Bilangan Prima	15
4	Proses Penukaran Koin dengan Pendekatan Greedy	19
5	Kompleksitas Waktu Algoritma Coin Exchange	21
6	Output Hasil Pengujian Algoritma Divide and Conquer	29
7	Trace Eksekusi Merge Sort D&C	31
8	Perbedaan Decrease and Conquer vs Divide and Conquer	34
9	Trace Rekursi Selection Sort D&C	36
10	Output Hasil Selection Sort Input Pengguna	37
11	Penelusuran BFS Manual dari Node 0	41
12	Penelusuran DFS Manual dari Node 0	41
13	Perbedaan Algoritma BFS dan DFS	42
14	Perbandingan Urutan Traversal BFS dan DFS	45
15	Validasi Hasil Program vs Manual pada Graf Pre Test	47

DAFTAR GAMBAR

1	Flowchart Algoritma Selection Sort	3
2	Flowchart Algoritma Insertion Sort	5
3	Flowchart Algoritma Pencarian Elemen Terbesar	11
4	Flowchart Algoritma Uji Bilangan Prima	13
5	Flowchart Algoritma Coin Exchange	18
6	Flowchart Algoritma Merge Sort	25
7	Flowchart Algoritma Quick Sort	26
8	Flowchart Algoritma Selection Sort Decrease and Conquer	35
9	Visualisasi Graf Pre Test (Gambar 6.4)	40
10	Visualisasi Graf Berarah Gambar 6.1 (Langkah Praktikum)	42
11	Flowchart Algoritma BFS	43
12	Flowchart Algoritma DFS	43
13	Urutan BFS dari Node 0 (angka dalam kurung = urutan kunjungan) .	46
14	Urutan DFS dari Node 0 (angka dalam kurung = urutan kunjungan) .	46

PRAKTIKUM 1

KOMPLEKSITAS ALGORITMA

1.1 Kompleksitas Algoritma

Kompleksitas algoritma adalah ukuran efisiensi suatu algoritma dilihat dari sumber daya yang dibutuhkan, yaitu waktu dan memori. Kompleksitas Waktu $T(n)$ mengukur jumlah operasi sebagai fungsi ukuran masukan n , sedangkan Kompleksitas Ruang $S(n)$ mengukur penggunaan memori. Analisis dilakukan pada tiga kondisi: best case (kondisi terbaik), worst case (kondisi terburuk), dan average case (kondisi rata-rata).

1.2 Notasi Big-O

Notasi Big-O menyatakan batas atas pertumbuhan kompleksitas suatu algoritma, mengabaikan konstanta dan suku tidak dominan. Hierarki kompleksitas dari yang paling efisien:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

1.3 Selection Sort

Definisi: Algoritma pengurutan yang bekerja dengan mencari elemen terkecil dari bagian array yang belum terurut, lalu menukarnya ke posisi yang tepat.

Cara Kerja: Setiap pass ke- i mencari elemen minimum dari $A[i..n-1]$ dan menukarnya dengan $A[i]$. Setelah pass ke- i , elemen $A[0..i]$ sudah pada posisi akhirnya.

Fungsi: Mengurutkan array secara ascending dengan pendekatan pemilihan elemen minimum.

Kelebihan: Sederhana, jumlah pertukaran minimal (maksimal $n-1$ kali), dan in-place.

Kekurangan: Kompleksitas $O(n^2)$ di semua kasus, tidak stabil, dan tidak adaptif.

1.4 Insertion Sort

Definisi: Algoritma pengurutan yang menyisipkan setiap elemen ke posisi yang tepat dalam bagian array yang sudah terurut, mirip cara menyortir kartu.

Cara Kerja: Mulai dari indeks $i=1$, setiap elemen $key=A[i]$ dibandingkan dengan

elemen di kirinya. Elemen yang lebih besar digeser ke kanan hingga posisi yang tepat ditemukan.

Fungsi: Mengurutkan array secara ascending dengan pendekatan penyisipan elemen.

Kelebihan: Efisien untuk data kecil atau hampir terurut (best case $O(n)$), stabil, adaptif, dan in-place.

Kekurangan: Worst case $O(n^2)$ dan banyak operasi pergeseran pada data acak berukuran besar.

Pertanyaan 1: Apa alat ukur sebuah algoritma dikatakan efisien menjadi solusi suatu permasalahan?

Jawaban: Alat ukur efisiensi algoritma ada dua: Kompleksitas Waktu $T(n)$ yang mengukur jumlah operasi komputasi sebagai fungsi ukuran masukan n , dan Kompleksitas Ruang $S(n)$ yang mengukur kebutuhan memori. Keduanya bersifat abstrak dan independen dari perangkat keras maupun compiler sehingga dapat digunakan sebagai acuan universal dalam membandingkan algoritma.

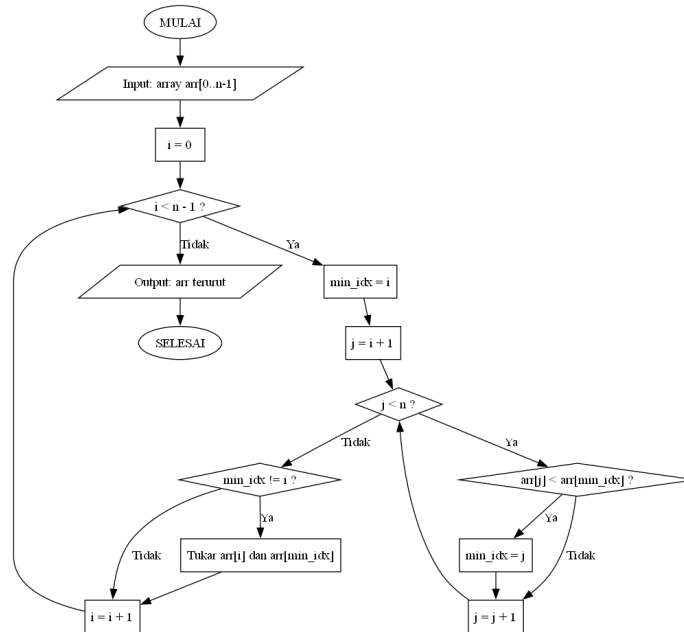
Pertanyaan 2: Mengapa kita perlu algoritma yang efisien?

Jawaban: Algoritma yang efisien diperlukan karena:

1. Selisih kompleksitas sangat terasa pada data besar. Untuk $n = 1.000.000$, algoritma $O(n)$ butuh 10^6 operasi sedangkan $O(n^2)$ butuh 10^{12} operasi.
2. Menghemat sumber daya komputasi berupa waktu proses, memori, dan energi.
3. Memungkinkan sistem yang skalabel karena performa tetap baik meskipun data bertambah.

1.5 Langkah Praktikum: Selection Sort

Flowchart



Gambar 1: Flowchart Algoritma Selection Sort

Source Code Python

```
def selection_sort(arr):
    """
    Mengurutkan array dengan algoritma Selection Sort.
    Setiap pass mencari elemen terkecil dari sisa array
    lalu menukarnya ke posisi yang tepat.
    """
    n = len(arr)
    total_comparisons = 0
    total_swaps = 0
    print(f"Array awal : {arr}")
    print("-" * 55)

    for i in range(n - 1):
        # Asumsikan elemen terkecil ada di posisi i
        min_idx = i
        pass_comparisons = 0
        pass_swaps = 0

        # Cari elemen terkecil dari i+1 hingga akhir array
        for j in range(i + 1, n):
            pass_comparisons += 1
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Tukar elemen terkecil ke posisi i (jika berbeda)
        if min_idx != i:
```



```

        arr[i], arr[min_idx] = arr[min_idx], arr[i]
        pass_swaps += 1

    total_comparisons += pass_comparisons
    total_swaps += pass_swaps

    print(f"Pass {i + 1:2d} | Array: {arr} | "
          f"Perbandingan: {pass_comparisons:2d} | "
          f"Pertukaran: {pass_swaps}")
    print("-" * 55)

print(f"Array akhir : {arr}")
print(f"Total perbandingan : {total_comparisons}")
print(f"Total pertukaran : {total_swaps}")
return arr

```

Analisis

Jawaban 1: Jumlah operasi perbandingan untuk setiap pass ke-i

Pada pass ke-i (i dari 0 sampai n-2), algoritma mencari elemen minimum dari posisi i+1 hingga n-1. Jumlah perbandingan pada setiap pass disajikan pada Tabel 1:

Tabel 1: Operasi Perbandingan Per Pass Selection Sort

Pass ke-i	Rentang Pencarian	Perbandingan
0	A[1] hingga A[4]	4
1	A[2] hingga A[4]	3
2	A[3] hingga A[4]	2
3	A[4] hingga A[4]	1

Jawaban 2: Jumlah operasi pertukaran elemen

Setiap pass melakukan paling banyak 1 kali pertukaran, hanya apabila elemen minimum (min_idx) bukan berada di posisi i. Pertukaran minimum 0 kali (array sudah terurut) dan maksimum n-1 kali. Pada Kasus Uji 1 dengan n=5, total pertukaran = 3 dari 4 pass.

Jawaban 3: Derivasi kompleksitas waktu

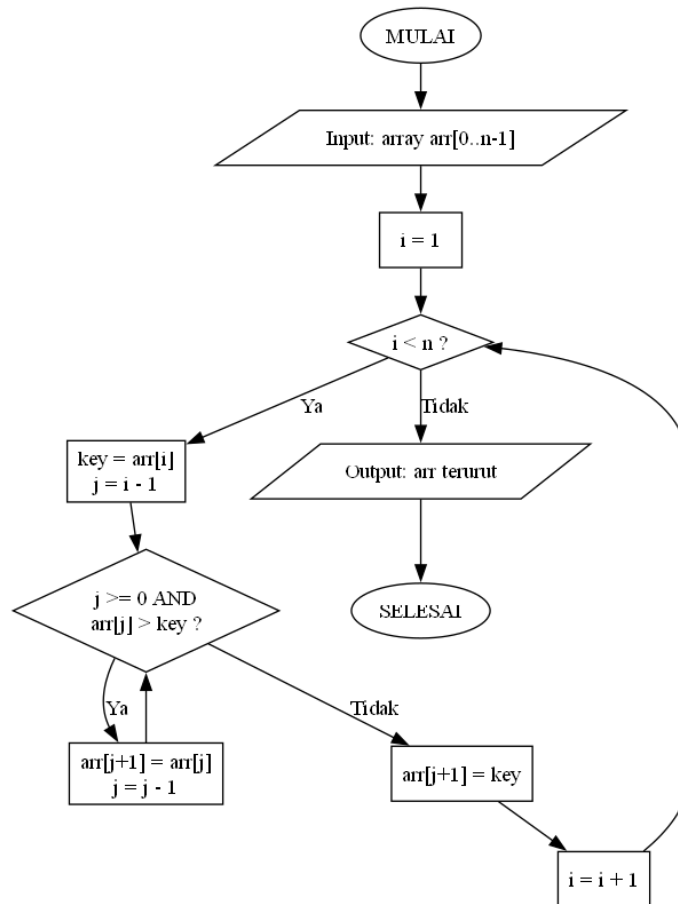
Total perbandingan dijumlahkan dari seluruh pass:

$$T(n) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2$$

Untuk n=5: $T(5) = 5 \times 4 / 2 = 10$, sesuai output program. Suku dominan adalah n^2 , sehingga kompleksitas waktu Selection Sort adalah $O(n^2)$.

1.6 Post Test: Insertion Sort

Flowchart



Gambar 2: Flowchart Algoritma Insertion Sort

Source Code Python

```
def insertion_sort(arr):
    """
    Mengurutkan array dengan algoritma Insertion Sort.
    Setiap elemen disisipkan ke posisi yang tepat
    di dalam bagian array yang sudah terurut.
    """
    n = len(arr)
    total_comparisons = 0
    total_shifts = 0
    print(f"Array awal : {arr}")
    print("-" * 60)

    for i in range(1, n):
        key = arr[i]
        j = i - 1
        pass_comparisons = 0
        pass_shifts = 0
```

```

# Geser elemen yang lebih besar dari key ke kanan
while j >= 0 and arr[j] > key:
    arr[j + 1] = arr[j]
    j -= 1
    pass_comparisons += 1
    pass_shifts += 1

# Hitung perbandingan terakhir (kondisi berhenti)
if j >= 0:
    pass_comparisons += 1

# Sisipkan key ke posisi yang tepat
arr[j + 1] = key
total_comparisons += pass_comparisons
total_shifts += pass_shifts

print(f"Pass {i:2d} | key={key:3d} | Array: {arr} | "
      f"Perbandingan: {pass_comparisons} | "
      f"Pergeseran: {pass_shifts}")
print("-" * 60)

print(f"Array akhir          : {arr}")
print(f"Total perbandingan   : {total_comparisons}")
print(f"Total pergeseran      : {total_shifts}")
return arr

```

Analisis

Jawaban 1: Kompleksitas waktu Insertion Sort

- **Best Case $O(n)$:** Array sudah terurut. Setiap elemen hanya dibandingkan sekali, tidak ada pergeseran. $T(n) = n-1$.
- **Worst Case $O(n^2)$:** Array terurut terbalik. Elemen ke- i harus digeser i kali. $T(n) = 1+2+\dots+(n-1) = n(n-1)/2$.
- **Average Case $O(n^2)$:** Rata-rata elemen ke- i melakukan $i/2$ perbandingan. $T(n) \approx n(n-1)/4$.

Jawaban 2: Perbandingan Selection Sort vs Insertion Sort Tabel 2 merangkum perbedaan karakteristik antara kedua algoritma pengurutan tersebut:

Tabel 2: Perbandingan Selection Sort dan Insertion Sort

Aspek	Insertion Sort	Selection Sort
Best Case	$O(n)$ — adaptif	$O(n^2)$ — tidak adaptif
Worst Case	$O(n^2)$	$O(n^2)$
Average Case	$O(n^2)$	$O(n^2)$
Pertukaran/Geser	Banyak (pergeseran)	Sedikit (maks. $n-1$)
Stabilitas	Stabil	Tidak Stabil
Adaptivitas	Adaptif	Tidak Adaptif

Kesimpulan: Insertion Sort lebih mangkus pada data yang hampir terurut karena bersifat adaptif dengan best case $O(n)$. Selection Sort lebih unggul dalam minimasi pertukaran (maks. $n-1$ swap), cocok ketika operasi swap memiliki biaya mahal. Pada data acak berukuran besar keduanya setara di $O(n^2)$.

1.7 Implementasi Nyata

Selection Sort dan Insertion Sort banyak diimplementasikan pada sistem dengan dataset kecil, seperti sistem pengolahan nilai akademik, antrian transaksi sederhana, atau pengurutan log pada embedded system. Dua jurnal berikut mendukung relevansi penggunaan kedua algoritma ini dalam praktik.

[1] **Penulis:** Karunanidhi, G. (2025)

Judul: "A Systematic Analysis on Performance and Computational Complexity of Sorting Algorithms"

Publikasi: Discover Computing, Vol. 28, No. 1, Hal. 250. Springer Nature. DOI: 10.1007/s10791-025-09724-w

Intisari: Review sistematis terhadap 12 teknik sorting klasik menggunakan analisis empiris pada MATLAB dengan ukuran data 100 hingga 100.000 elemen. Hasil menunjukkan bahwa selection sort mengalami penurunan performa signifikan pada data besar karena kompleksitas $O(n^2)$ yang konsisten di semua kasus. Sebaliknya, insertion sort terbukti lebih efisien untuk data berukuran kecil berkat sifat adaptifnya.

Link: <https://link.springer.com/article/10.1007/s10791-025-09724-w>

[2] Penulis: Halili, F. dan Dika, E. (2021)

Judul: "Results of Comparison Between Different Sorting Algorithms"

Publikasi: IBU International Journal of Technical and Natural Sciences, Vol. 2, Hal. 39-52. International Balkan University Press.

Intisari: Studi komparatif lima algoritma pengurutan berdasarkan waktu eksekusi menggunakan C++. Hasil empiris menunjukkan bahwa Insertion Sort merupakan pilihan yang baik untuk dataset berukuran kecil hingga beberapa ribu elemen karena sifat adaptif (best case $O(n)$) dan kemudahan implementasinya. Insertion Sort tidak disarankan untuk list dengan lebih dari beberapa ribu elemen karena kompleksitas kuadratnya.

Link: <https://ijtns.ibupress.com/uploads/pdf/22.pdf>

Kelebihan di konteks dataset kecil:

- Implementasi sederhana dan mudah dipelihara.
- Tidak memerlukan memori tambahan (in-place).
- Insertion Sort efisien untuk data yang ditambahkan secara bertahap (inkremental).

Kekurangan di konteks dataset kecil:

- Tidak cocok untuk $n > 10.000$ karena kompleksitas $O(n^2)$ menjadi bottleneck.
- Untuk kebutuhan skala besar, disarankan beralih ke Merge Sort atau Quick Sort dengan kompleksitas $O(n \log n)$.

PRAKTIKUM 2

ALGORITMA BRUTE FORCE

2.1 Algoritma Brute Force

Definisi: Brute Force adalah pendekatan lempang (straightforward) untuk memecahkan suatu persoalan dengan cara yang paling langsung dan jelas (obvious way), tanpa memanfaatkan struktur khusus dari masalah yang diselesaikan.

Cara Kerja: Setiap kemungkinan solusi dieksplorasi satu per satu secara sistematis hingga solusi yang benar ditemukan atau seluruh kemungkinan habis diperiksa.

Fungsi: Digunakan sebagai solusi dasar (baseline) untuk masalah yang belum memiliki algoritma khusus, atau ketika ukuran masukan cukup kecil sehingga kompleksitas tinggi masih dapat ditoleransi.

Kelebihan: Mudah dipahami dan diimplementasikan, selalu menghasilkan solusi yang benar apabila solusi memang ada, dan tidak memerlukan analisis matematis yang rumit.

Kekurangan: Kompleksitas waktu umumnya tinggi ($O(n^2)$ atau lebih), tidak efisien untuk data besar, dan tidak memanfaatkan struktur masalah untuk mempercepat pencarian.

2.2 Perpangkatan

Menghitung a^n ($a > 0$, n bilangan bulat tak-negatif) dengan cara mengalikan 1 dengan a sebanyak n kali secara berurutan. Ciri Brute Force: tidak menggunakan teknik fast exponentiation atau divide and conquer, melainkan perkalian berulang satu per satu.

2.3 Pencarian Elemen Terbesar

Mencari elemen terbesar dari array berisi n bilangan bulat dengan membandingkan setiap elemen dengan nilai maksimum sementara. Ciri Brute Force: setiap elemen diperiksa satu per satu tanpa pengecualian. Kompleksitas $T(n) = n - 1$ perbandingan, sehingga $O(n)$.

2.4 Perkalian Dua Matriks

Menghitung hasil perkalian dua matriks A dan B berukuran $n \times n$ dengan menghitung setiap elemen $C[i][j]$ satu per satu melalui tiga perulangan bersarang (i, j, k).

Ciri Brute Force: tidak ada optimasi seperti Strassen. Kompleksitas $O(n^3)$.

2.5 Uji Bilangan Prima

Definisi: Bilangan prima adalah bilangan yang hanya habis dibagi oleh 1 dan dirinya sendiri.

Ciri Brute Force: membagi x dengan setiap bilangan dari 2 sampai $\sqrt{x}$ satu per satu. Jika tidak ada yang habis membagi, maka x adalah bilangan prima. Kompleksitas worst case $O(\sqrt{n})$.

Pertanyaan 1: Jelaskan apa ciri khas dari penyelesaian persoalan dengan algoritma Brute Force!

Jawaban: Ciri khas Brute Force adalah penyelesaian masalah secara langsung, sederhana, dan jelas (straightforward/obvious way) tanpa memanfaatkan struktur atau sifat khusus dari masalah. Setiap kemungkinan diperiksa satu per satu secara menyeluruh. Pendekatan ini menjamin solusi yang benar namun umumnya menghasilkan kompleksitas yang tinggi karena tidak ada optimasi.

Pertanyaan 2: Jelaskan ciri khas algoritma Brute Force pada contoh berikut:

a. Algoritma Perpangkatan: Brute Force terlihat dari cara menghitung a^n dengan mengalikan a sebanyak n kali secara berulang (hasil = hasil * a), tanpa memanfaatkan teknik seperti fast exponentiation. Kompleksitas $O(n)$.

b. Algoritma Pencarian Elemen Terbesar: Brute Force terlihat dari pembandingan setiap elemen $A[k]$ dengan nilai maks satu per satu dari awal hingga akhir array. Tidak ada strategi pemangkasan. Kompleksitas $O(n)$.

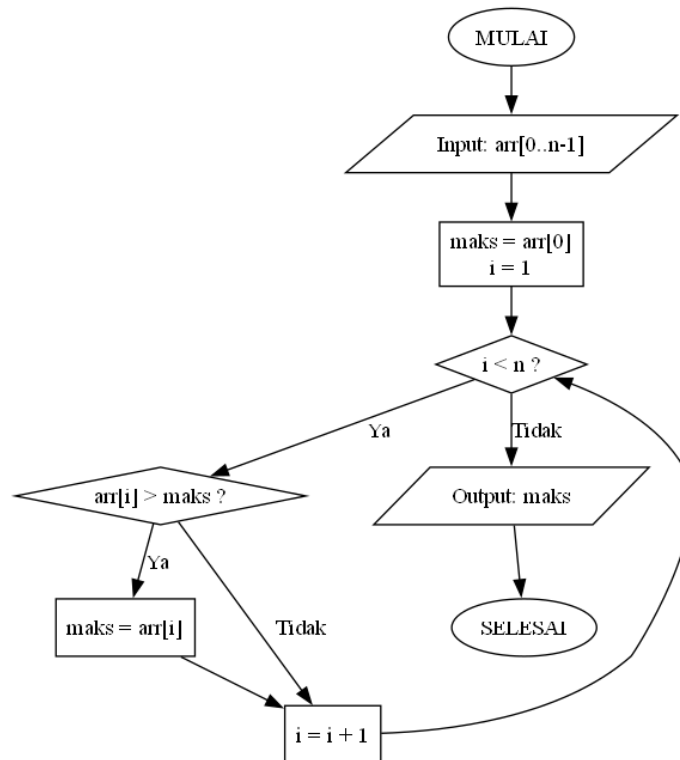
c. Algoritma Perkalian Dua Matriks: Brute Force terlihat dari tiga perulangan bersarang (i, j, k) yang menghitung setiap elemen $C[i][j]$ satu per satu tanpa optimasi. Kompleksitas $O(n^3)$.

d. Algoritma Uji Bilangan Prima: Brute Force terlihat dari pembagian x dengan setiap bilangan dari 2 hingga $\sqrt{x}$ secara berurutan. Tidak ada penggunaan sifat bilangan prima untuk mempercepat. Kompleksitas $O(\sqrt{n})$.

2.6 Langkah Praktikum

(a) Pencarian Elemen Terbesar

Flowchart



Gambar 3: Flowchart Algoritma Pencarian Elemen Terbesar

Source Code Python

```
def cari_element_terbesar(arr):
    """
    Mencari elemen terbesar dalam array menggunakan Brute Force.
    Strategi: bandingkan setiap elemen dengan nilai maksimum sementara.
    Kompleksitas: T(n) = n - 1 perbandingan -> O(n)
    """
    n = len(arr)
    maks = arr[0] # asumsikan elemen pertama adalah terbesar
    idx_maks = 0
    print(f"Array : {arr}")
    print(f"Inisialisasi : maks = {maks} (indeks 0)")
    print("-" * 55)
    for k in range(1, n):
        # bandingkan setiap elemen dengan nilai maks saat ini
        if arr[k] > maks:
            maks = arr[k]
            idx_maks = k
            status = f"UPDATE maks = {maks}"
        else:
            status = "tidak ada perubahan"
        print(f"k={k} | A[k]={arr[k]:4d} | maks={maks:4d} | {status}")
    print("-" * 55)
    print(f"Elemen terbesar : {maks} (indeks {idx_maks})")
    print(f"Jumlah perbandingan : {n - 1} --> T(n) = n - 1 = O(n) ")
```



```

    return maks

# =====
# Main Program
# =====
if __name__ == "__main__":
    # Kasus Uji 1: Array acak
    data1 = [3, 7, 1, 9, 4, 6, 2]
    print("=" * 55)
    print(" PENCARIAN ELEMEN TERBESAR - Kasus Uji 1 (acak)")
    print("=" * 55)
    cari_elemen_terbesar(data1)

    # Kasus Uji 2: Terbesar di awal (best case perbaruan)
    data2 = [15, 3, 8, 1, 10]
    print("\n" + "=" * 55)
    print(" PENCARIAN ELEMEN TERBESAR - Kasus Uji 2 (terbesar di awal)")
    print("=" * 55)
    cari_elemen_terbesar(data2)

    # Kasus Uji 3: Terbesar di akhir (worst case perbaruan)
    data3 = [2, 5, 8, 11, 20]
    print("\n" + "=" * 55)
    print(" PENCARIAN ELEMEN TERBESAR - Kasus Uji 3 (terbesar di akhir)")
    print("=" * 55)
    cari_elemen_terbesar(data3)

```

Analisis Output

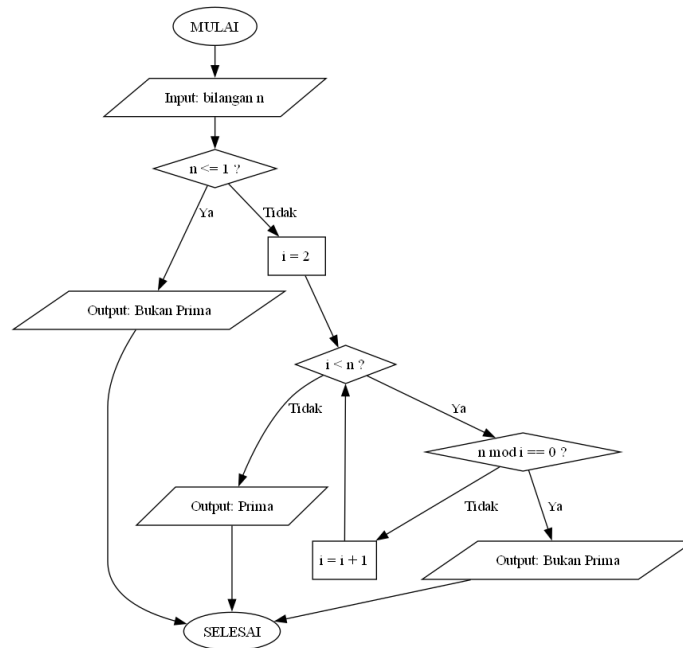
Program diuji dengan tiga kasus:

- **Kasus 1 (acak)** Array [3, 7, 1, 9, 4, 6, 2]: elemen terbesar = 9 ditemukan pada indeks 3. Jumlah perbandingan = 6 ($n-1 = 7-1$). Terjadi dua kali UPDATE maks, yaitu saat $k=1$ (nilai 7) dan $k=3$ (nilai 9).
- **Kasus 2 (terbesar di awal)** Array [15, 3, 8, 1, 10]: elemen terbesar = 15 langsung ditemukan di inisialisasi. Tidak ada UPDATE maks selama iterasi. Jumlah perbandingan tetap = 4 ($n-1$).
- **Kasus 3 (terbesar di akhir)** Array [2, 5, 8, 11, 20]: maks diperbarui di setiap iterasi karena data terurut menaik. Jumlah perbandingan = 4 ($n-1$).

Ketiga kasus menunjukkan bahwa jumlah perbandingan selalu $n-1$, tidak bergantung pada posisi elemen terbesar. Ini membuktikan $T(n) = n - 1$ sehingga $O(n)$.

(b) Uji Bilangan Prima

Flowchart



Gambar 4: Flowchart Algoritma Uji Bilangan Prima

Source Code Python

```
import math

def is_prima(x):
    """
    Menguji apakah x adalah bilangan prima menggunakan Brute Force.
    Strategi: bagi x dengan setiap bilangan dari 2 sampai sqrt(x).
    Jika tidak ada yang habis membagi, maka x adalah bilangan prima.
    Kompleksitas:
    Worst case : T(n) = sqrt(n) - 1 --> O(sqrt(n))
    Best case : T(n) = 1 --> O(1) (genap atau < 2)
    """
    if x < 2:
        return False # 1 dan bilangan negatif bukan prima
    if x == 2:
        return True # 2 adalah prima, kasus khusus
    if x % 2 == 0:
        return False # bilangan genap selain 2 bukan prima

    y = int(math.sqrt(x)) # batas atas pembagi
    k = 3 # mulai dari 3, langkah 2 (hanya bilangan ganjil)

    while k <= y:
        if x % k == 0:
            return False # ditemukan pembagi, bukan prima
        k += 2

    return True # tidak ada pembagi ditemukan, x adalah prima
```

Analisis Output

Program mencetak semua bilangan prima dari 1 hingga 50: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47] sebanyak 15 bilangan.

Pengujian verbose pada beberapa angka:

- **x = 17 (prima):** $\text{sqrt}(17) = 4$, hanya 1 langkah ($17 \% 3 = 2$), langsung dinyatakan prima.
- **x = 18 (bukan prima):** langsung dieliminasi di pengecekan bilangan genap tanpa masuk loop.
- **x = 37 (prima):** $\text{sqrt}(37) = 6$, 2 langkah ($\% 3$ dan $\% 5$), dinyatakan prima.
- **x = 49 (bukan prima):** $\text{sqrt}(49) = 7$, pada langkah ke-3 ditemukan $49 \% 7 = 0$, bukan prima.

2.7 Post Test

Jawaban 1: Kompilasi dan eksekusi program

Program dijalankan menggunakan Python 3 dan menghasilkan output yang sesuai dengan pseudocode modul. Pencarian elemen terbesar menghasilkan hasil yang konsisten dengan $T(n) = n-1$ perbandingan di setiap kasus uji, sedangkan uji bilangan prima berhasil mengidentifikasi 15 bilangan prima dalam rentang 1 hingga 50 dengan benar.

Jawaban 2: Implementasi Python pencarian elemen terbesar

Implementasi Python dari pseudocode modul terdapat pada file `pencarian_terbesar.py` dengan fungsi `cari_element_terbesar(arr)`. Kode mengonversi prosedur CariElemenTerbesar dari pseudocode ke Python 3 dengan penambahan pelacakan jumlah perbandingan dan tampilan status tiap iterasi (lihat source code pada 3.1a).

Jawaban 3: Kompleksitas algoritma uji bilangan prima

Analisis kompleksitas dilakukan berdasarkan jumlah iterasi pembagian:

- **Best Case $O(1)$:** Terjadi ketika $x < 2$, $x = 2$, atau x adalah bilangan genap ($x \% 2 = 0$). Tidak masuk loop, hanya 1 operasi pengecekan.
- **Worst Case $O(\sqrt{n})$:** Terjadi ketika x adalah bilangan prima. Loop berjalan dari $k=3$ hingga $k=\text{sqrt}(x)$ dengan langkah 2. Jumlah iterasi maksimum = $(\text{sqrt}(x) - 3)/2 + 1$, yang bersifat $O(\sqrt{n})$.

Tabel 3: Kompleksitas Operasi Uji Bilangan Prima

Bilangan x	sqrt(x)	Iterasi Maksimum	Hasil
17	4	1	Prima
37	6	2	Prima
97	9	4	Prima
49	7	3	Bukan Prima (49 % 7 = 0)

Karena iterasi dibatasi oleh $\sqrt{x}$, maka kompleksitas waktu uji bilangan prima adalah $T(n) = O(\sqrt{n})$. Ini jauh lebih efisien dibandingkan pendekatan Brute Force murni yang mencoba semua pembagi dari 2 hingga $x-1$ dengan kompleksitas $O(n)$.

2.8 Implementasi Nyata

Algoritma Brute Force seperti pencarian elemen terbesar dan uji bilangan prima digunakan secara nyata dalam berbagai bidang, mulai dari kriptografi hingga sistem keamanan jaringan. Jurnal berikut mendukung relevansinya.

[1] **Penulis:** Baur, K. et al. (2023)

Judul: "Comparing and Reviewing Modern Primality Tests"

Publikasi: ResearchGate, 2023. Tersedia di: <https://www.researchgate.net/publication/369685477>

Intisari: Makalah ini membandingkan berbagai algoritma uji bilangan prima mulai dari Brute Force trial division ($O(\sqrt{n})$) hingga algoritma modern seperti Miller-Rabin dan AKS. Brute Force digunakan sebagai baseline referensi karena kesederhanaan dan ketepatannya. Hasilnya menunjukkan bahwa Brute Force masih relevan untuk bilangan kecil, namun tidak skalabel untuk bilangan kriptografis berukuran besar (ratusan digit).

Link: https://www.researchgate.net/publication/369685477_Comparing_and_Reviewing_Modern_Primality_Tests

Kelebihan dalam konteks nyata:

- Implementasi sederhana dan mudah diverifikasi kebenarannya.
- Cocok sebagai baseline perbandingan sebelum menggunakan algoritma yang lebih kompleks.
- Uji bilangan prima $O(\sqrt{n})$ sudah cukup efisien untuk kebutuhan pembuatan

kunci pada sistem kriptografi skala kecil.

Kekurangan dalam konteks nyata:

- Tidak skalabel: untuk bilangan prima kriptografis (ratusan digit), kompleksitas $O(\sqrt{n})$ masih terlalu lambat dan perlu diganti algoritma probabilistik seperti Miller-Rabin.
- Pencarian elemen terbesar $O(n)$ memang sudah optimal untuk kasus linear, namun tidak berlaku jika data terstruktur (misal sudah terurut) yang memungkinkan pencarian lebih cepat.

PRAKTIKUM 3

ALGORITMA GREEDY

3.1 Algoritma Greedy

Definisi: Algoritma Greedy adalah metode yang paling populer untuk memecahkan persoalan optimasi. Prinsipnya adalah "take what you can get now" yaitu pada setiap langkah selalu dipilih keputusan terbaik secara lokal dengan harapan menghasilkan solusi global yang optimal.

Cara Kerja: Solusi dibangun langkah per langkah (step by step). Pada setiap langkah, dari banyak pilihan yang tersedia, dipilih satu pilihan terbaik berdasarkan kriteria tertentu tanpa mempertimbangkan konsekuensi di langkah berikutnya.

Fungsi: Digunakan untuk menyelesaikan persoalan optimasi minimasi maupun maksimasi, seperti penukaran uang, penjadwalan, Huffman encoding, dan shortest path.

Kelebihan: Implementasi mudah, efisien dalam waktu eksekusi, dan menghasilkan solusi optimal pada sistem koin kanonik (seperti sistem mata uang nyata).

Kekurangan: Tidak selalu menghasilkan solusi global yang optimal untuk semua kasus. Pada sistem koin non-kanonik, solusi greedy bisa lebih banyak koin dibanding solusi optimal yang didapat melalui dynamic programming.

3.2 Masalah Penukaran Uang (Coin Exchange)

Diberikan nilai uang A yang ingin ditukar dan himpunan koin $C = \{d_1, d_2, \dots, d_n\}$. Tujuannya adalah menemukan himpunan koin S sehingga total nilai $S = A$ dengan jumlah koin semimumimum mungkin.

Fungsi objektif: Minimisasi $F = \sum x_i$ (jumlah koin yang digunakan)

Kendala: $\sum d_i \times x_i = A$ (total nilai koin harus tepat sama dengan A)

Strategi Greedy: Pada setiap langkah, pilih koin dengan nilai terbesar dari himpunan koin yang tersisa selama masih tidak melebihi sisa nilai yang perlu ditukar.

Pertanyaan: Jelaskan apa yang Anda ketahui tentang algoritma Greedy!

Jawaban:

Algoritma Greedy adalah strategi perancangan algoritma yang membangun solusi secara bertahap dengan selalu memilih pilihan terbaik yang tersedia pada setiap

langkah (locally optimal choice). Strategi ini berprinsip bahwa dengan terus memilih yang terbaik secara lokal, akan diperoleh solusi terbaik secara global.

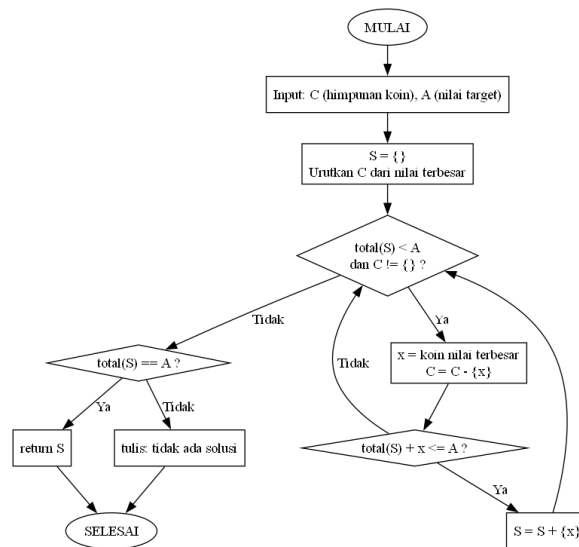
Greedy diterapkan pada persoalan optimasi yang memiliki dua sifat utama:

1. **Greedy Choice Property:** solusi global optimal dapat dicapai dengan membuat pilihan lokal yang optimal.
2. **Optimal Substructure:** solusi optimal dari masalah mengandung solusi optimal dari sub-masalahnya.

Contoh persoalan yang dapat diselesaikan dengan Greedy antara lain penukaran uang dengan sistem mata uang kanonik, algoritma Kruskal dan Prim untuk minimum spanning tree, algoritma Dijkstra untuk shortest path, serta Huffman Coding untuk kompresi data. Kelemahannya, Greedy tidak selalu menghasilkan solusi optimal apabila kedua sifat di atas tidak terpenuhi.

3.3 Langkah Praktikum: Coin Exchange

Flowchart



Gambar 5: Flowchart Algoritma Coin Exchange

Source Code Python

```
def coin_exchange(coins, amount):
    # urutkan koin dari nilai terbesar ke terkecil (strategi greedy)
    coins_sorted = sorted(coins, reverse=True)
    result = []
    remaining = amount

    for coin in coins_sorted:
```

```

# ambil koin sebanyak mungkin selama tidak melebihi sisa
while remaining >= coin:
    result.append(coin)
    remaining -= coin

if remaining == 0:
    return result
else:
    return None # tidak ada solusi

```

Analisis Output

Program diuji dengan koin standar [1000, 500, 200, 100, 50, 25, 10, 5, 1] dan nilai target 2750:

Tabel 4: Proses Penukaran Koin dengan Pendekatan Greedy

Langkah	Koin Dipilih	Sisa Nilai
1	1000	1750
2	1000	750
3	500	250
4	200	50
5	50	0

Hasil: koin yang digunakan = [1000, 1000, 500, 200, 50], total 5 koin. Algoritma greedy berhasil menemukan solusi minimal karena sistem koin Rupiah bersifat kanonik, artinya pilihan koin terbesar di setiap langkah selalu mengarah ke solusi dengan jumlah koin minimum.

3.4 Post Test

Source Code Python

```

def analisis_kasus(coins, amount, label=""):
    print(f"\n{'='*50}")
    if label:
        print(f" {label}")
    print(f"{'='*50}")
    print(f"Nilai ditukar : {amount}")
    print(f"Koin tersedia : {sorted(coins, reverse=True)}")

    hasil = coin_exchange(coins, amount)

    if hasil:
        rekap = Counter(hasil)
        print(f"Solusi          : {dict(rekap)}")

```



```

        print(f"Jumlah koin    : {len(hasil)}")
    else:
        print("Tidak ada solusi (greedy gagal).")

    return hasil

def hitung_kompleksitas(coins, amount):
    # hitung iterasi aktual untuk analisis kompleksitas
    coins_sorted = sorted(coins, reverse=True)
    n = len(coins_sorted)
    iterasi = 0
    remaining = amount

    for coin in coins_sorted:
        while remaining >= coin:
            remaining -= coin
            iterasi += 1

    print(f"\nAnalisis Kompleksitas (amount={amount}, n={n} jenis koin):")
    print(f"Jumlah iterasi while : {iterasi}")
    print(f"Jumlah iterasi for    : {n}")
    print(f"T(n) = O(n + amount/koin_terkecil) -> praktis O(n) untuk denominasi wajar")

if __name__ == "__main__":
    koin_std = [1000, 500, 200, 100, 50, 25, 10, 5, 1]

    # kasus 1: solusi optimal
    analisis_kasus(koin_std, 2750, "Kasus 1: Solusi Normal")

    # kasus 2: jumlah besar
    analisis_kasus(koin_std, 9875, "Kasus 2: Jumlah Besar")

    # kasus 3: greedy tidak optimal (denominasi tidak kanonik)
    koin_non_kanonik = [10, 6, 1]
    analisis_kasus(koin_non_kanonik, 12, "Kasus 3: Greedy Tidak Optimal (koin [10,6,1], target 12)")
    print("  Solusi greedy: 10+1+1 = 3 koin")
    print("  Solusi optimal: 6+6    = 2 koin")

    # analisis kompleksitas
    hitung_kompleksitas(koin_std, 2750)

```

Jawaban 1: Implementasi, output, dan analisis kasus

Program diuji pada tiga kasus:

- **Kasus 1 (nilai 2750, koin kanonik):** Solusi = { 1000:2, 500:1, 200:1, 50:1 }, total 5 koin. Greedy optimal.
- **Kasus 2 (nilai 9875, koin kanonik):** Solusi = { 1000:9, 500:1, 200:1, 100:1, 50:1, 25:1 }, total 14 koin. Greedy optimal.
- **Kasus 3 (nilai 12, koin [10,6,1] non-kanonik):** Solusi greedy = { 10:1, 1:2 }

= 3 koin. Solusi optimal = {6:2} = 2 koin. Greedy tidak optimal karena sistem koin tidak kanonik.

Kasus 3 membuktikan bahwa algoritma Greedy tidak menjamin solusi optimal secara universal. Untuk sistem koin non-kanonik, diperlukan pendekatan Dynamic Programming.

Jawaban 2: Kompleksitas waktu

Analisis kompleksitas dilakukan berdasarkan dua komponen utama:

- **Loop for iterasi jenis koin:** dilakukan sebanyak n kali (n = jumlah jenis koin). Kompleksitas $O(n)$.
- **Loop while pengambilan koin:** untuk setiap jenis koin, jumlah pengambilan bergantung pada nilai koin dan sisa amount. Dalam kasus terburuk total iterasi while = $\text{floor}(A / \text{koin_terkecil})$.

Sehingga total kompleksitas:

$$T(n) = O(n + A/d_{\min})$$

dengan n = jumlah jenis koin dan $d_{\min}$ = nilai koin terkecil. Untuk sistem mata uang praktis ($d_{\min} = 1$), kasus terburuk adalah $T(n) = O(A)$. Namun secara praktis karena koin terbesar mendominasi pengambilan, jumlah iterasi while jauh lebih kecil dari A sehingga kompleksitas efektif mendekati $O(n)$.

Tabel 5: Kompleksitas Waktu Algoritma Coin Exchange

Komponen	Kompleksitas
Pengurutan koin (sorting)	$O(n \log n)$
Iterasi jenis koin (for)	$O(n)$
Pengambilan koin per jenis (while)	$O(A / d_{\min})$ worst case
Total keseluruhan	$O(n \log n + A / d_{\min})$

3.5 Implementasi Nyata

Algoritma Greedy pada masalah coin change diterapkan secara nyata dalam sistem transaksi cryptocurrency, khususnya pada proses coin selection pada dompet digital berbasis UTXO (Unspent Transaction Output).

[1] **Penulis:** Wei, X., Wu, C., Yu, H., Luo, X., dan Huang, S. (2023)

Judul: "A Coin Selection Strategy Based on the Greedy and Genetic Algorithm"

Publikasi: Complex and Intelligent Systems, Vol. 9, Hal. 421-434. Springer Nature. DOI: 10.1007/s40747-022-00799-2

Intisari: Makalah ini membahas penerapan Greedy pada proses coin selection di dompet Bitcoin berbasis UTXO. Metode greedy digunakan sebagai komponen utama untuk memilih set koin yang mendekati nilai target transaksi, kemudian dikombinasikan dengan algoritma genetika untuk meminimalkan biaya transaksi dan mengurangi akumulasi UTXO bernilai kecil (dust). Hasil eksperimen menunjukkan strategi greedy-genetika mengungguli metode greedy murni dalam efisiensi jaringan jangka panjang, meski greedy murni tetap lebih cepat untuk transaksi individual.

Link: <https://link.springer.com/article/10.1007/s40747-022-00799-2>

Relevansi dengan Praktikum:

Seperti halnya penukaran uang konvensional, coin selection pada Bitcoin menggunakan prinsip yang sama yaitu memilih koin dengan nilai terbesar terlebih dahulu untuk meminimalkan jumlah koin yang dipakai. Kompleksitas $O(n)$ pada greedy murni membuatnya sangat cepat untuk transaksi real-time.

Kelebihan dalam konteks ini:

- Waktu eksekusi sangat cepat, cocok untuk sistem transaksi real-time yang membutuhkan respons instan.
- Implementasi sederhana dan deterministik sehingga mudah diaudit dan diverifikasi.
- Pada sistem koin kanonik (mata uang resmi maupun Bitcoin), greedy menghasilkan solusi yang optimal atau mendekati optimal.

Kekurangan dalam konteks ini:

- Tidak optimal untuk sistem koin non-kanonik: greedy bisa menghasilkan lebih banyak koin dari yang seharusnya.
- Tidak mempertimbangkan dampak jangka panjang seperti akumulasi dust UTXO yang memperbesar beban jaringan, sehingga perlu dikombinasikan dengan metode lain seperti algoritma genetika.

PRAKTIKUM 4

ALGORITMA DIVIDE & CONQUER

4.1 Algoritma Divide and Conquer

Definisi: Divide and Conquer adalah paradigma perancangan algoritma yang memecah masalah berukuran n menjadi beberapa upa-masalah yang lebih kecil, menyelesaikan setiap upa-masalah secara rekursif, lalu menggabungkan solusinya menjadi solusi masalah semula.

Tiga Tahap:

1. **Divide:** membagi masalah menjadi beberapa upa-masalah dengan ukuran lebih kecil dan karakteristik yang sama.
2. **Conquer:** menyelesaikan setiap upa-masalah secara rekursif hingga mencapai basis (ukuran cukup kecil untuk diselesaikan langsung).
3. **Combine:** menggabungkan solusi upa-masalah menjadi solusi masalah semula.

Fungsi: Digunakan untuk menyelesaikan persoalan yang dapat dibagi secara alami menjadi sub-masalah serupa, seperti pengurutan, pencarian minimum-maksimum, perkalian matriks, dan fast Fourier transform.

Kelebihan: Kompleksitas lebih efisien dibanding pendekatan iteratif biasa (contoh: Merge Sort $O(n \log n)$ vs $O(n^2)$), paralel secara alami karena upa-masalah independen, dan solusinya elegan secara rekursif.

Kekurangan: Overhead rekursi (stack calls) untuk masalah berukuran kecil, penggunaan memori tambahan pada tahap combine (contoh: Merge Sort membutuhkan $O(n)$ memori tambahan).

4.2 MinMaks Divide and Conquer

Mencari nilai minimum dan maksimum sekaligus pada tabel berisi n elemen dengan membagi tabel menjadi dua bagian di titik tengah, mencari MinMaks setiap bagian secara rekursif, lalu membandingkan hasil keduanya. Kompleksitas: $T(n) = O(n)$.

4.3 Merge Sort

Algoritma pengurutan berbasis D&C dengan tiga langkah: (a) Divide: bagi tabel menjadi dua bagian berukuran $n/2$, (b) Conquer: urutkan tiap bagian secara rekursif,

(c) Merge: gabungkan dua bagian terurut menjadi satu tabel terurut. Kompleksitas: $T(n) = O(n \log n)$ di semua kasus. Bersifat stabil.

4.4 Quick Sort dan Partisi

Algoritma pengurutan D&C dengan pendekatan hard split/easy join. Tabel dipartisi menggunakan elemen pivot sehingga elemen di kiri pivot lebih kecil dan di kanan lebih besar, kemudian setiap bagian diurutkan secara rekursif tanpa tahap combine eksplisit. Kompleksitas: average $O(n \log n)$, worst case $O(n^2)$ apabila pivot selalu ekstrem. Tidak stabil.

Pertanyaan 1: Buatlah fungsi QuickSort!

```
def quick_sort(arr, i, j):
    # basis: array 1 elemen atau kosong
    if i >= j:
        return
    # partisi dan dapatkan indeks pembagi
    k = partisi(arr, i, j)
    # rekursif urutkan bagian kiri dan kanan
    quick_sort(arr, i, k)
    quick_sort(arr, k + 1, j)

if __name__ == "__main__":
    data = [4, 12, 23, 9, 21, 1, 35, 2, 24]
    print("Data awal:", data)

    # MinMaks D&C
    mn, mx = minmaks(data, 0, len(data) - 1)
    print(f"\nMinMaks D&C -> Min: {mn}, Maks: {mx}")

    # Merge Sort
    arr_ms = data.copy()
    merge_sort(arr_ms, 0, len(arr_ms) - 1)
    print(f"Merge Sort -> {arr_ms}")

    # Quick Sort
    arr_qs = data.copy()
    quick_sort(arr_qs, 0, len(arr_qs) - 1)
    print(f"Quick Sort -> {arr_qs}")
```

Pertanyaan 2: Buatlah fungsi Partition!

```
def partisi(arr, i, j):
    # pilih elemen tengah sebagai pivot
    pivot = arr[(i + j) // 2]
    p, q = i, j
    while True:
        # scan dari kiri sampai menemukan elemen >= pivot
        while arr[p] < pivot:
            p += 1
```

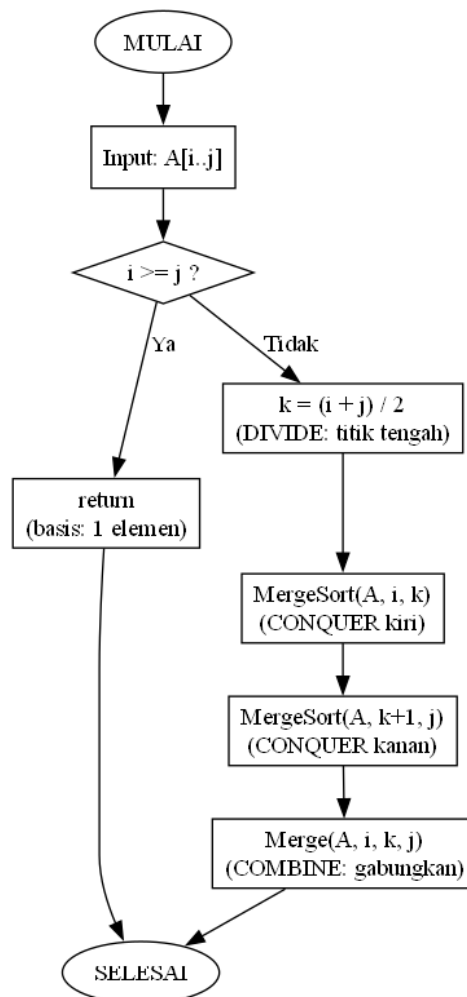
```

# scan dari kanan sampai menemukan elemen <= pivot
while arr[q] > pivot:
    q -= 1
# jika belum bertemu, tukar dan geser indeks
if p < q:
    arr[p], arr[q] = arr[q], arr[p]
    p += 1
    q -= 1
else:
    break
return q

```

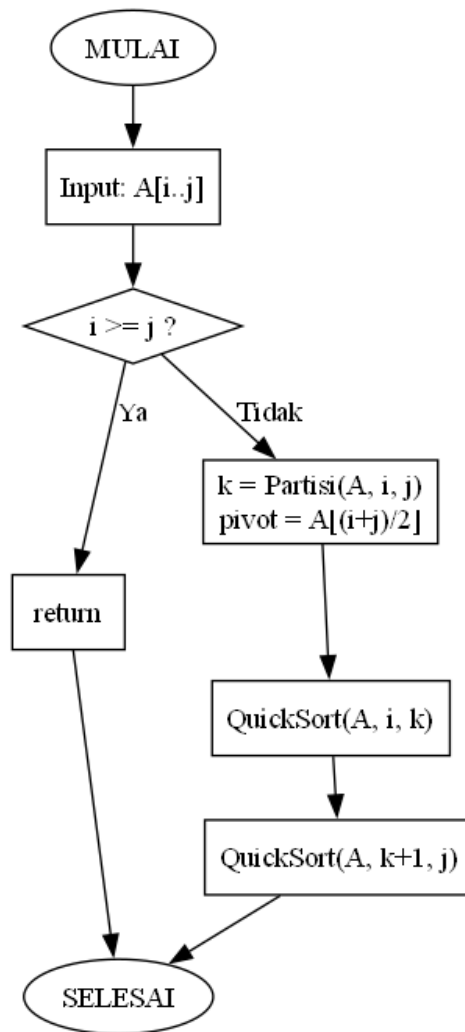
4.5 Langkah Praktikum: Program Sorting

Flowchart Merge Sort



Gambar 6: Flowchart Algoritma Merge Sort

Flowchart Quick Sort



Gambar 7: Flowchart Algoritma Quick Sort

Source Code Python: MinMaks D&C

```
def minmaks(arr, i, j):  
    # basis: 1 elemen  
    if i == j:  
        return arr[i], arr[i]  
  
    # basis: 2 elemen, bandingkan langsung  
    if i == j - 1:  
        if arr[i] < arr[j]:  
            return arr[i], arr[j]  
        else:  
            return arr[j], arr[i]  
  
    # rekursif: bagi tabel di titik tengah  
    k = (i + j) // 2  
    min1, maks1 = minmaks(arr, i, k)
```

```
min2, maks2 = minmaks(arr, k + 1, j)

# combine: ambil min dan maks dari kedua bagian
return min(min1, min2), max(maks1, maks2)
```

Source Code Python: Merge Sort

```
def merge(arr, kiri, tengah, kanan):
    # salin dua bagian ke array sementara
    kidal1 = arr[kiri:tengah + 1]
    kidal2 = arr[tengah + 1:kanan + 1]

    i = j = 0
    k = kiri

    # gabungkan dua bagian secara terurut
    while i < len(kidal1) and j < len(kidal2):
        if kidal1[i] <= kidal2[j]:
            arr[k] = kidal1[i]
            i += 1
        else:
            arr[k] = kidal2[j]
            j += 1
        k += 1

    # salin sisa elemen kiri jika ada
    while i < len(kidal1):
        arr[k] = kidal1[i]
        i += 1
        k += 1

    # salin sisa elemen kanan jika ada
    while j < len(kidal2):
        arr[k] = kidal2[j]
        j += 1
        k += 1

def merge_sort(arr, i, j):
    # basis: array 1 elemen sudah terurut
    if i >= j:
        return

    # divide: tentukan titik tengah
    k = (i + j) // 2

    # conquer: urutkan bagian kiri dan kanan secara rekursif
    merge_sort(arr, i, k)
    merge_sort(arr, k + 1, j)

    # combine: gabungkan hasil pengurutan
    merge(arr, i, k, j)
```

Source Code Python: Quick Sort


```

def partisi(arr, i, j):
    # pilih elemen tengah sebagai pivot
    pivot = arr[(i + j) // 2]
    p, q = i, j

    while True:
        # scan dari kiri sampai menemukan elemen >= pivot
        while arr[p] < pivot:
            p += 1
        # scan dari kanan sampai menemukan elemen <= pivot
        while arr[q] > pivot:
            q -= 1

        # jika belum bertemu, tukar dan geser indeks
        if p < q:
            arr[p], arr[q] = arr[q], arr[p]
            p += 1
            q -= 1
        else:
            break

    return q

def quick_sort(arr, i, j):
    # basis: array 1 elemen atau kosong
    if i >= j:
        return

    # partisi dan dapatkan indeks pembagi
    k = partisi(arr, i, j)

    # rekursif urutkan bagian kiri dan kanan
    quick_sort(arr, i, k)
    quick_sort(arr, k + 1, j)

if __name__ == "__main__":
    data = [4, 12, 23, 9, 21, 1, 35, 2, 24]
    print("Data awal:", data)

    # MinMaks D&C
    mn, mx = minmaks(data, 0, len(data) - 1)
    print(f"\nMinMaks D&C -> Min: {mn}, Maks: {mx}")

    # Merge Sort
    arr_ms = data.copy()
    merge_sort(arr_ms, 0, len(arr_ms) - 1)
    print(f"Merge Sort -> {arr_ms}")

    # Quick Sort
    arr_qs = data.copy()
    quick_sort(arr_qs, 0, len(arr_qs) - 1)
    print(f"Quick Sort -> {arr_qs}")

```

Analisis Output

Program dijalankan dengan data awal [4, 12, 23, 9, 21, 1, 35, 2, 24]:

Tabel 6: Output Hasil Pengujian Algoritma Divide and Conquer

Algoritma	Output	Keterangan
MinMaks D&C	Min: 1, Maks: 35	Ditemukan dengan $T(n) = O(n)$ perbandingan
Merge Sort	[1, 2, 4, 9, 12, 21, 23, 24, 35]	Terurut stabil, $O(n \log n)$
Quick Sort	[1, 2, 4, 9, 12, 21, 23, 24, 35]	Terurut, rata-rata $O(n \log n)$

MinMaks D&C: Tabel dibagi dua pada $k=4$. Bagian kiri [4,12,23,9,21] menghasilkan $\text{min}=4$, $\text{maks}=23$. Bagian kanan [1,35,2,24] menghasilkan $\text{min}=1$, $\text{maks}=35$. Setelah combine: $\text{min}=1$, $\text{maks}=35$.

Merge Sort: Pembagian berlangsung rekursif hingga setiap bagian berukuran 1 elemen, lalu digabung secara terurut. Karena bersifat stabil, urutan relatif elemen bernilai sama terjaga.

Quick Sort: Pivot dipilih dari elemen tengah setiap iterasi. Proses partisi memindahkan elemen lebih kecil ke kiri dan lebih besar ke kanan pivot, kemudian kedua bagian diurutkan rekursif.

4.6 Post Test: Merge Sort Standalone

Source Code Python

```
def merge(arr, kiri, tengah, kanan):
    # salin dua bagian ke array sementara
    bagian_kiri = arr[kiri:tengah + 1]
    bagian_kanan = arr[tengah + 1:kanan + 1]

    i = j = 0
    k = kiri

    # gabungkan dua bagian ke array utama secara terurut
    while i < len(bagian_kiri) and j < len(bagian_kanan):
        if bagian_kiri[i] <= bagian_kanan[j]:
            arr[k] = bagian_kiri[i]
            i += 1
        else:
            arr[k] = bagian_kanan[j]
            j += 1
        k += 1

    # salin sisa bagian kiri jika masih ada
```

```

while i < len(bagian_kiri):
    arr[k] = bagian_kiri[i]
    i += 1
    k += 1

# salin sisa bagian kanan jika masih ada
while j < len(bagian_kanan):
    arr[k] = bagian_kanan[j]
    j += 1
    k += 1

def merge_sort(arr, i, j, depth=0):
    indent = "  " * depth
    # basis: array dengan 1 elemen sudah terurut
    if i >= j:
        return

    # divide: hitung titik tengah pembagi
    k = (i + j) // 2
    print(f"{indent}Divide [{i}..{j}] -> kiri [{i}..{k}], kanan [{k+1}..{j}]")

    # conquer: urutkan bagian kiri secara rekursif
    merge_sort(arr, i, k, depth + 1)

    # conquer: urutkan bagian kanan secara rekursif
    merge_sort(arr, k + 1, j, depth + 1)

    # combine: gabungkan hasil pengurutan kedua bagian
    merge(arr, i, k, j)
    print(f"{indent}Combine [{i}..{j}] -> {arr[i:j+1]}")

if __name__ == "__main__":
    data = [38, 27, 43, 3, 9, 82, 10]
    print("Data awal      :", data)
    print("\nProses Merge Sort (D&C):")
    print("-" * 45)

    merge_sort(data, 0, len(data) - 1)

    print("-" * 45)
    print("Data terurut    :", data)

```

Jawaban 1: Trace proses Divide-Conquer-Combine

Berikut trace verbose output program dengan data [38, 27, 43, 3, 9, 82, 10]:

Tabel 7: Trace Eksekusi Merge Sort D&C

Langkah	Operasi	Hasil
1	Divide [0..6] -> kiri [0..3], kanan [4..6]	
2	Divide [0..3] -> kiri [0..1], kanan [2..3]	
3	Divide [0..1] -> kiri [0..0], kanan [1..1]	
4	Combine [0..1]	[27, 38]
5	Divide [2..3] -> kiri [2..2], kanan [3..3]	
6	Combine [2..3]	[3, 43]
7	Combine [0..3]	[3, 27, 38, 43]
8	Divide [4..5] -> kiri [4..4], kanan [5..5]	
9	Combine [4..5]	[9, 82]
10	Combine [4..6]	[9, 10, 82]
11	Combine [0..6]	[3, 9, 10, 27, 38, 43, 82]

Jawaban 2: Output eksekusi

Data awal: [38, 27, 43, 3, 9, 82, 10]. Setelah proses Merge Sort D&C selesai, data terurut menjadi: [3, 9, 10, 27, 38, 43, 82]. Seluruh proses pembagian (Divide) dan penggabungan (Combine) ditampilkan verbose sehingga alur rekursi terlihat jelas dari kedalaman indentasi output.

4.7 Implementasi Nyata

Algoritma Merge Sort berbasis Divide and Conquer banyak diimplementasikan pada sistem pengurutan data berskala besar, termasuk pada sistem pemrosesan paralel. Jurnal berikut mendukung relevansi penerapannya.

[1] **Penulis:** Ketchaya, S. dan Rattanatanurak, A. (2023)

Judul: "Parallel Multi-Deque Partition Dual-Deque Merge Sorting Algorithm Using OpenMP"

Publikasi: Scientific Reports, Vol. 13, Nature Publishing Group. DOI: 10.1038/s41598-023-33583-4

Intisari: Makalah ini mengusulkan algoritma pengurutan paralel berbasis Divide and Conquer yang dikembangkan dari Quick Sort dan Merge Sort menggunakan OpenMP pada sistem memori bersama. Algoritma MPDMSort terbukti lebih efisien dibandingkan implementasi sekuensial standar dalam pengurutan data biologis,

saintifik, dan big data. Ini membuktikan bahwa prinsip D&C yang memecah masalah menjadi upa-masalah independen secara inheren mendukung paralelisme dan skalabilitas.

Link: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10115789/>

Kelebihan dalam konteks nyata:

- Kompleksitas $O(n \log n)$ yang konsisten membuat Merge Sort andal untuk pengurutan dataset besar seperti log server, data transaksi, dan rekaman medis.
- Sifat stabil Merge Sort menjamin urutan relatif elemen bernilai sama terjaga, penting dalam pengurutan multi-kunci.
- Prinsip D&C secara alami mendukung paralelisasi sehingga cocok untuk sistem multi-core modern.

Kekurangan dalam konteks nyata:

- Merge Sort membutuhkan memori tambahan $O(n)$ untuk proses penggabungan, sehingga kurang ideal untuk sistem dengan memori terbatas.
- Quick Sort memiliki worst case $O(n^2)$ apabila pemilihan pivot tidak tepat, sehingga perlu strategi pemilihan pivot yang baik seperti median-of-three untuk data dunia nyata.

PRAKTIKUM 5

ALGORITMA DECREASE & CONQUER

5.1 Algoritma Decrease and Conquer

Definisi: Decrease and Conquer adalah metode perancangan algoritma yang mereduksi persoalan berukuran n menjadi beberapa subpersoalan yang lebih kecil, tetapi selanjutnya hanya memproses satu subpersoalan saja secara rekursif. Berbeda dengan Divide and Conquer yang memproses semua subpersoalan dan menggabungkan seluruh solusinya, Decrease and Conquer tidak memiliki tahap combine.

Dua Tahap:

1. **Decrease:** mereduksi ukuran masalah menjadi subpersoalan yang lebih kecil, biasanya dengan mengurangi satu elemen atau membagi setengah.
2. **Conquer:** memproses satu subpersoalan secara rekursif hingga mencapai basis penyelesaian langsung.

Tiga Varian:

1. **Decrease by a constant:** ukuran masalah berkurang sebesar konstanta tetap setiap iterasi (biasanya 1). Contoh: Selection Sort, Insertion Sort, pencarian sekuensial.
2. **Decrease by a constant factor:** ukuran masalah berkurang sebesar faktor tetap setiap iterasi (biasanya 2). Contoh: Binary Search, Fake Coin Problem.
3. **Decrease by a variable size:** ukuran pengurangan bervariasi tiap iterasi. Contoh: BST search, Selection Problem.

Fungsi: Digunakan untuk menyelesaikan persoalan yang dapat disederhanakan secara bertahap dengan mereduksi ukuran masalah, seperti pengurutan, pencarian, dan traversal.

Kelebihan: Lebih sederhana dari D&C karena tidak memerlukan tahap combine, overhead rekursi lebih kecil, dan cocok untuk masalah yang solusinya dapat ditemukan dengan mereduksi satu elemen setiap langkah.

Kekurangan: Tidak seefisien D&C untuk masalah yang secara alami dapat diparalelkan, dan kompleksitas bisa tetap $O(n^2)$ apabila strategi pereduksian tidak optimal.

5.2 Selection Sort sebagai Decrease and Conquer

Selection Sort adalah contoh klasik varian Decrease by a Constant ($k=1$). Pada setiap langkah, masalah pengurutan array $A[i..j]$ direduksi menjadi submasalah $A[i+1..j]$ dengan cara meletakkan elemen terkecil di posisi i melalui fungsi Bagi. Tidak ada tahap combine, karena posisi $A[i]$ sudah final setelah fungsi Bagi dijalankan.

Kompleksitas waktu: $T(n) = T(n-1) + (n-1)$ sehingga $T(n) = n(n-1)/2 = O(n^2)$.

Pertanyaan 1: Apakah karakteristik dari algoritma Decrease and Conquer?

Jawaban: Karakteristik utama Decrease and Conquer adalah: (1) masalah berukuran n direduksi menjadi satu submasalah yang lebih kecil, (2) hanya satu submasalah yang diproses secara rekursif, (3) tidak ada tahap combine karena solusi submasalah langsung menjadi bagian dari solusi masalah asli, dan (4) proses pereduksian berlangsung secara bertahap hingga mencapai basis kasus yang dapat diselesaikan langsung.

Pertanyaan 2: Jelaskan perbedaan algoritma Decrease and Conquer dengan Divide and Conquer!

Tabel 8: Perbedaan Decrease and Conquer vs Divide and Conquer

Aspek	Decrease and Conquer	Divide and Conquer
Pembagian submasalah	Menjadi submasalah lebih kecil	Menjadi dua atau lebih submasalah
Jumlah submasalah diproses	Hanya satu submasalah	Semua submasalah diproses
Tahap Combine	Tidak ada	Ada, hasil digabungkan
Ukuran pereduksian	Constant, factor, atau variable	Biasanya $n/2$ atau n/k
Contoh algoritma	Selection Sort, Binary Search	Merge Sort, Quick Sort, MinMaks

Pertanyaan 3: Sebutkan algoritma selain Selection Sort yang menggunakan teknik Decrease and Conquer!

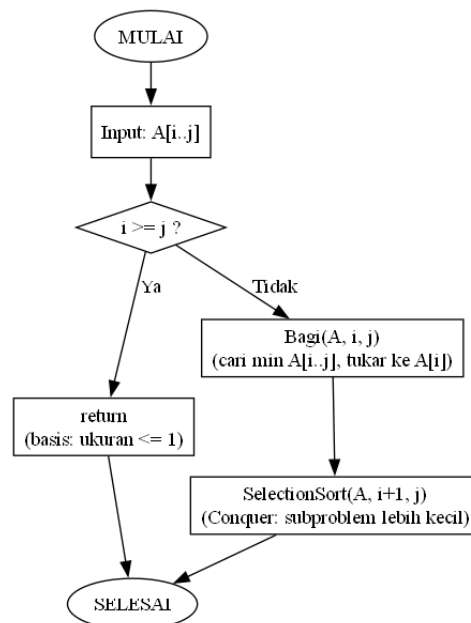
Jawaban:

- **Insertion Sort** (decrease by constant 1): setiap langkah menyisipkan satu elemen ke subarray yang sudah terurut.

- **Binary Search** (decrease by constant factor 2): setiap langkah mengeliminasi setengah ruang pencarian berdasarkan perbandingan dengan elemen tengah.
- **Topological Sort** (decrease by constant 1): setiap langkah memilih satu simpul tanpa dependensi masuk dan menghapusnya dari graf.
- **BST Search** (decrease by variable size): setiap langkah memilih subtree kiri atau kanan berdasarkan perbandingan nilai.
- **Euclid GCD** (decrease by variable size): setiap langkah mengganti pasangan (m, n) dengan $(n, m \bmod n)$ hingga remainder = 0.

5.3 Langkah Praktikum: Selection Sort D&C

Flowchart



Gambar 8: Flowchart Algoritma Selection Sort Decrease and Conquer

Source Code Python

```

def bagi(arr, i, j):
    # cari indeks elemen terkecil dari A[i..j]
    idx_min = i
    for k in range(i + 1, j + 1):
        if arr[k] < arr[idx_min]:
            idx_min = k

    # tukar elemen terkecil ke posisi i
    arr[i], arr[idx_min] = arr[idx_min], arr[i]

def selection_sort_dc(arr, i, j):

```



```

# basis: ukuran subarray <= 1, sudah terurut
if i >= j:
    return

# decrease: letakkan elemen terkecil A[i..j] ke posisi i
bagi(arr, i, j)

# conquer: urutkan sisa subarray A[i+1..j] secara rekursif
selection_sort_dc(arr, i + 1, j)

if __name__ == "__main__":
    arr = [64, 25, 12, 22, 11]
    print("Data awal      :", arr)

    selection_sort_dc(arr, 0, len(arr) - 1)

    print("Data terurut:", arr)

```

Analisis Output

Program dijalankan dengan data awal [64, 25, 12, 22, 11] sesuai modul:

Tabel 9: Trace Rekursi Selection Sort D&C

Rekursi ke	Subarray A[i..j]	Elemen Min	Setelah Bagi	Subproblem
1	A[0..4] = [64,25,12,22,11]	11 (idx 4)	[11,25,12,22,64]	A[1..4]
2	A[1..4] = [25,12,22,64]	12 (idx 2)	[11,12,25,22,64]	A[2..4]
3	A[2..4] = [25,22,64]	22 (idx 3)	[11,12,22,25,64]	A[3..4]
4	A[3..4] = [25,64]	25 (idx 3)	[11,12,22,25,64]	A[4..4]
5	A[4..4] = [64]	basis i>=j	tidak ada tukar	selesai

Output akhir: [11, 12, 22, 25, 64]. Setiap rekursi mereduksi submasalah sebesar 1 elemen (decrease by constant = 1). Tidak ada tahap combine karena posisi A[i] sudah final setelah setiap pemanggilan Bagi.

5.4 Post Test: Selection Sort dengan Input Pengguna

Source Code Python

```

def bagi(arr, i, j):
    # cari indeks elemen terkecil dari A[i..j]
    idx_min = i
    for k in range(i + 1, j + 1):
        if arr[k] < arr[idx_min]:
            idx_min = k

    # tukar elemen terkecil ke posisi i

```

```

        arr[i], arr[idx_min] = arr[idx_min], arr[i]

def selection_sort_dc(arr, i, j):
    # basis: ukuran subarray <= 1, sudah terurut
    if i >= j:
        return

    # decrease: letakkan elemen terkecil ke posisi i
    bagi(arr, i, j)

    # conquer: rekursif pada subarray yang lebih kecil
    selection_sort_dc(arr, i + 1, j)

if __name__ == "__main__":
    # terima input array dari pengguna
    raw = input("Masukkan bilangan dipisah spasi: ")
    arr = list(map(int, raw.split()))

    print("Data awal      :", arr)
    selection_sort_dc(arr, 0, len(arr) - 1)
    print("Data terurut:", arr)

```

Analisis Output

Program menerima input dari pengguna berupa bilangan yang dipisah spasi, mengonversinya ke list integer, lalu menjalankan Selection Sort D&C. Contoh eksekusi dengan input "5 3 8 1 9 2":

Tabel 10: Output Hasil Selection Sort Input Pengguna

Input	Data Awal	Data Terurut
5 3 8 1 9 2	[5, 3, 8, 1, 9, 2]	[1, 2, 3, 5, 8, 9]

Implementasi ini membuktikan bahwa Selection Sort berbasis Decrease and Conquer menghasilkan output yang identik dengan versi iteratif, dengan alur rekursi yang lebih eksplisit mencerminkan prinsip decrease by a constant.

5.5 Implementasi Nyata

Pendekatan Decrease and Conquer pada Selection Sort telah dikaji secara akademis untuk penerapan paralel pada sistem komputasi modern. Jurnal berikut mendukung relevansinya.

[1] **Penulis:** Ganapathi, P. dan Chowdhury, R. (2022)

Judul: "Parallel Divide-and-Conquer Algorithms for Bubble Sort, Selection Sort and Insertion Sort"

Publikasi: The Computer Journal, Vol. 65, No. 10, Hal. 2709-2719. Oxford University Press. DOI: 10.1093/comjnl/bxab107

Intisari: Makalah ini menyajikan algoritma rekursif berbasis Decrease and Conquer untuk Selection Sort, Bubble Sort, dan Insertion Sort yang diparalelkan menggunakan Intel Cilk Plus dengan 32 thread. Hasil eksperimen menunjukkan Selection Sort rekursif D&C mencapai speedup hingga ratusan kali dibanding versi sekuensialnya pada input terurut terbalik, membuktikan bahwa struktur D&C secara inheren mendukung paralelisme meskipun kompleksitas asimtotiknya tetap $O(n^2)$.

Link: <https://academic.oup.com/comjnl/article/65/10/2709/6334046>

Kelebihan dalam konteks nyata:

- Struktur rekursif Decrease and Conquer secara alami mendukung paralelisasi, memungkinkan speedup signifikan pada sistem multi-core untuk data berukuran sedang.
- Implementasi rekursif lebih mudah dibuktikan kebenarannya secara formal dibanding versi iteratif, penting dalam sistem kritis seperti avionik atau medis.
- Cocok untuk dataset kecil hingga menengah pada embedded system yang memerlukan algoritma sederhana dengan footprint memori minimal.

Kekurangan dalam konteks nyata:

- Kompleksitas $O(n^2)$ tetap tidak berubah meskipun direkursifkan, sehingga tidak disarankan untuk dataset besar dibanding Merge Sort atau Quick Sort.
- Overhead pemanggilan fungsi rekursif membutuhkan stack memori tambahan, yang perlu diperhatikan pada sistem dengan memori terbatas.

PRAKTIKUM 6

ALGORITMA BFS DAN DFS

6.1 Traversal Graf

Traversal graf adalah proses mengunjungi setiap simpul (node) dalam sebuah graf secara sistematis. Terdapat dua pendekatan representasi graf dalam traversal: graf statis (sudah terbentuk sebelum pencarian, direpresentasikan sebagai struktur data) dan graf dinamis (terbentuk selama proses pencarian berlangsung). Algoritma traversal dikategorikan sebagai uninformed/blind search karena tidak menggunakan informasi heuristik untuk memandu pencarian.

6.2 Breadth First Search (BFS)

Definisi: BFS adalah algoritma traversal graf yang mengunjungi semua simpul secara berlapis (level by level), dimulai dari simpul awal v , kemudian seluruh tetangganya, lalu tetangga dari tetangganya, dan seterusnya.

Cara Kerja: Menggunakan struktur data antrian (queue). Simpul awal dimasukkan ke antrian, lalu setiap simpul yang dikeluarkan dari depan antrian akan memasukkan semua tetangganya yang belum dikunjungi ke belakang antrian.

Fungsi: Menemukan jalur terpendek pada graf tidak berbobot, eksplorasi jaringan sosial, web crawling, dan level-order traversal pada pohon.

Kelebihan: Menjamin jalur terpendek pada graf tidak berbobot, mengeksplorasi graf secara menyeluruh per level, dan tidak berisiko stack overflow.

Kekurangan: Membutuhkan memori $O(V)$ untuk antrian, tidak efisien untuk graf sangat lebar karena antrian bisa sangat besar, dan tidak efisien untuk mencari jalur dalam pada graf besar.

6.3 Depth First Search (DFS)

Definisi: DFS adalah algoritma traversal graf yang mengeksplorasi setiap cabang sedalam mungkin sebelum backtrack ke simpul sebelumnya dan mencoba cabang lain, dimulai dari simpul awal v .

Cara Kerja: Menggunakan rekursi (atau stack). Dari simpul yang dikunjungi, langsung masuk ke tetangga pertama yang belum dikunjungi, terus ke dalam hingga tidak ada tetangga baru, baru kemudian backtrack.

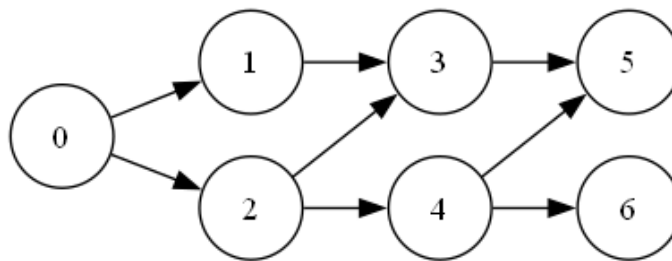
Fungsi: Deteksi siklus dalam graf, topological sorting, penyelesaian puzzle dan labirin, pencarian komponen terhubung, dan path finding pada graf berarah.

Kelebihan: Hemat memori $O(H)$ di mana H adalah kedalaman rekursi, efisien untuk eksplorasi jalur dalam, dan mudah diimplementasikan secara rekursif.

Kekurangan: Tidak menjamin jalur terpendek, berisiko stack overflow pada graf sangat dalam, dan dapat terjebak dalam siklus jika tidak ada penandaan simpul yang dikunjungi.

Pertanyaan 1: Analisis graf berarah pada Gambar 6.4 dan tuliskan urutan jalurnya secara manual apabila menggunakan algoritma BFS dan DFS mulai dari simpul/node ke-0!

Graf Gambar 6.4 (7 simpul): Edge: $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 5$, $4 \rightarrow 5$, $4 \rightarrow 6$



Gambar 9: Visualisasi Graf Pre Test (Gambar 6.4)

Jawaban BFS manual dari node 0:

Tabel 11: Penelusuran BFS Manual dari Node 0

Langkah	Node Diproses	Antrian Setelah	Alasan
1	0	[1, 2]	kunjungi 0, masukkan tetangga 1 dan 2
2	1	[2, 3]	kunjungi 1, masukkan tetangga 3
3	2	[3, 4]	kunjungi 2, masukkan 4 (3 sudah di antrian)
4	3	[4, 5]	kunjungi 3, masukkan tetangga 5
5	4	[5, 6]	kunjungi 4, masukkan 6 (5 sudah di antrian)
6	5	[6]	kunjungi 5, tidak ada tetangga baru
7	6	[]	kunjungi 6, tidak ada tetangga baru

Urutan BFS: 0, 1, 2, 3, 4, 5, 6

Jawaban DFS manual dari node 0:

Tabel 12: Penelusuran DFS Manual dari Node 0

Langkah	Node	Aksi
1	0	kunjungi 0, rekursi ke tetangga pertama: 1
2	1	kunjungi 1, rekursi ke tetangga: 3
3	3	kunjungi 3, rekursi ke tetangga: 5
4	5	kunjungi 5, tidak ada tetangga, backtrack ke 3, ke 1, ke 0
5	2	dari 0 rekursi ke tetangga berikutnya: 2
6	4	kunjungi 2, rekursi ke 4 (3 dan 5 sudah dikunjungi)
7	6	kunjungi 4, rekursi ke 6 (5 sudah dikunjungi)

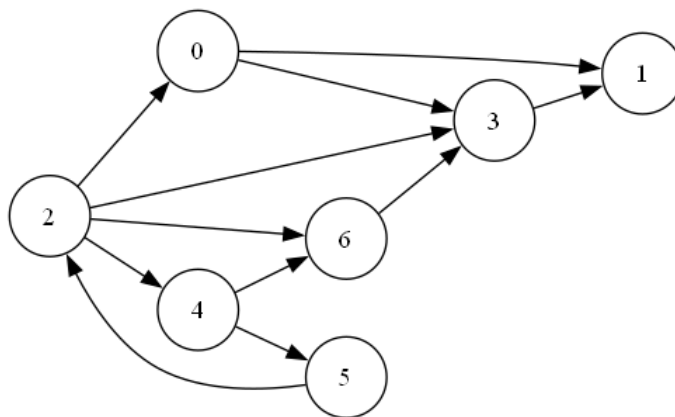
Urutan DFS: 0, 1, 3, 5, 2, 4, 6

Pertanyaan 2: Apa perbedaan utama antara algoritma BFS dan DFS?

Tabel 13: Perbedaan Algoritma BFS dan DFS

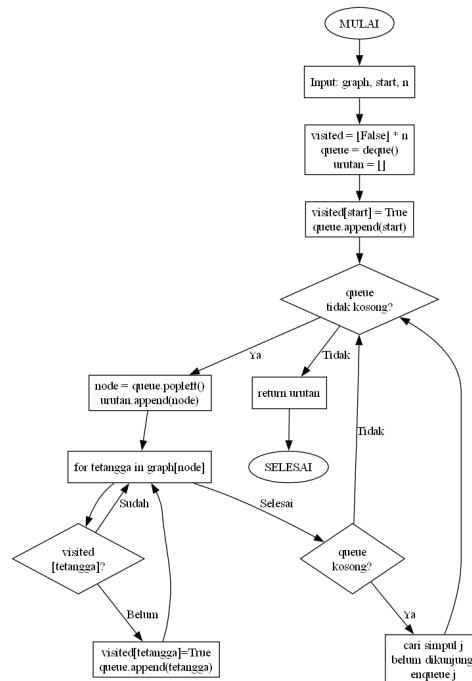
Aspek	BFS	DFS
Strategi penelusuran	Melebar (level by level)	Mendalam (cabang habis du- lu)
Struktur data	Queue (antrian)	Stack / Rekursi
Kompleksitas me- mori	$O(V)$ untuk antrian	$O(H)$, H = kedalaman rekursi
Jalur terpendek	Menjamin pada graf tidak berbobot	Tidak menjamin
Penggunaan utama	Shortest path, web crawling	Deteksi siklus, topological sort
Kompleksitas waktu	$O(V + E)$	$O(V + E)$

6.4 Langkah Praktikum: BFS dan DFS pada Graf 6.1



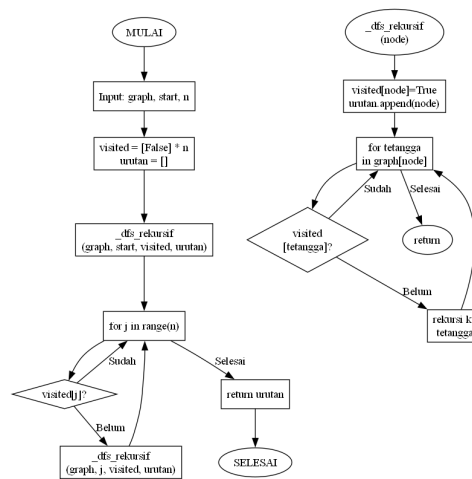
Gambar 10: Visualisasi Graf Berarah Gambar 6.1 (Langkah Praktikum)

Flowchart BFS



Gambar 11: Flowchart Algoritma BFS

Flowchart DFS



Gambar 12: Flowchart Algoritma DFS

Source Code Python: BFS

```
def bfs(graph, start, n):
    visited = [False] * n
    urutan = []
    queue = deque()
```



```

# tandai simpul awal sebagai dikunjungi dan masukkan ke antrian
visited[start] = True
queue.append(start)

while queue:
    # ambil simpul paling depan antrian
    node = queue.popleft()
    urutan.append(node)

    # kunjungi semua tetangga yang belum dikunjungi
    for tetangga in graph[node]:
        if not visited[tetangga]:
            visited[tetangga] = True
            queue.append(tetangga)

    # jika antrian kosong tapi masih ada simpul yang belum dikunjungi
    if not queue:
        for j in range(n):
            if not visited[j]:
                visited[j] = True
                queue.append(j)
                break

return urutan

if __name__ == "__main__":
    # graf berarah Gambar 6.1 sesuai kode tester modul
    n = 7
    graph = {
        0: [1, 3],
        1: [],
        2: [0, 3, 4, 6],
        3: [1],
        4: [5, 6],
        5: [2],
        6: [3],
    }
    hasil = bfs(graph, 0, n)
    print("BFS mulai dari node 0:", hasil)

```

Source Code Python: DFS

```

def _dfs_rekursif(graph, node, visited, urutan):
    # tandai simpul saat ini sebagai dikunjungi
    visited[node] = True
    urutan.append(node)

    # kunjungi semua tetangga yang belum dikunjungi secara rekursif
    for tetangga in graph[node]:
        if not visited[tetangga]:
            _dfs_rekursif(graph, tetangga, visited, urutan)

def dfs(graph, start, n):
    visited = [False] * n
    urutan = []

```

```

# mulai DFS dari simpul awal
_dfs_rekursif(graph, start, visited, urutan)

# tangani simpul yang belum terjangkau
for j in range(n):
    if not visited[j]:
        _dfs_rekursif(graph, j, visited, urutan)

return urutan

if __name__ == "__main__":
    # graf berarah Gambar 6.1 sesuai kode tester modul
    n = 7
    graph = {
        0: [1, 3],
        1: [],
        2: [0, 3, 4, 6],
        3: [1],
        4: [5, 6],
        5: [2],
        6: [3],
    }
    hasil = dfs(graph, 0, n)
    print("DFS mulai dari node 0:", hasil)

```

Analisis Output dan Perbedaan Kedua Algoritma

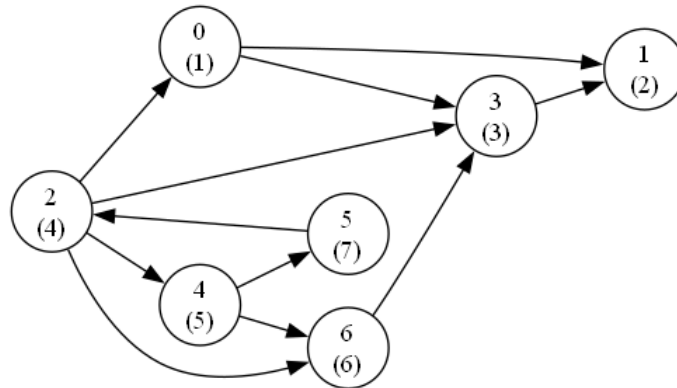
Program dijalankan pada graf berarah Gambar 6.1 (7 simpul) dimulai dari node 0. Graf memiliki edge: $0 \rightarrow 1$, $0 \rightarrow 3$, $2 \rightarrow 0$, $2 \rightarrow 3$, $2 \rightarrow 4$, $2 \rightarrow 6$, $3 \rightarrow 1$, $4 \rightarrow 5$, $4 \rightarrow 6$, $5 \rightarrow 2$, $6 \rightarrow 3$.

Tabel 14: Perbandingan Urutan Traversal BFS dan DFS

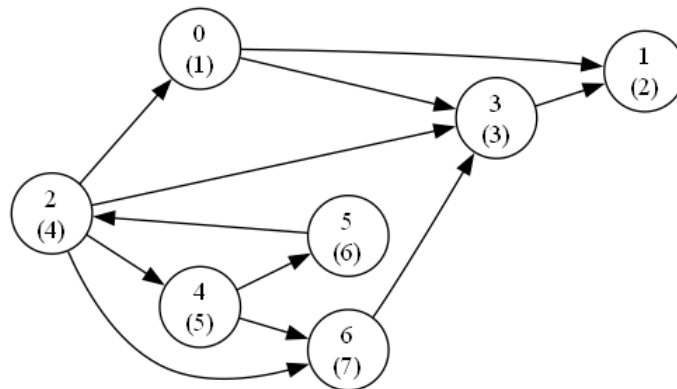
Algoritma	Urutan Traversal	Simpul ke-5 di-kunjungi ke-	Simpul ke-6 di-kunjungi ke-
BFS	[0, 1, 3, 2, 4, 6, 5]	ke-7 (terakhir)	ke-6
DFS	[0, 1, 3, 2, 4, 5, 6]	ke-6	ke-7 (terakhir)

Perbedaan terlihat pada urutan kunjungan simpul 5 dan 6. BFS mengunjungi simpul 6 sebelum 5 karena 6 masuk antrian lebih dulu (saat memproses simpul 4, edge $4 \rightarrow 6$ diperiksa sebelum $4 \rightarrow 5$ jika indeks lebih kecil diutamakan). DFS sebaliknya, masuk ke simpul 5 terlebih dahulu mengikuti cabang $4 \rightarrow 5$ sebelum backtrack ke $4 \rightarrow 6$.

Visualisasi Urutan Traversal



Gambar 13: Urutan BFS dari Node 0 (angka dalam kurung = urutan kunjungan)



Gambar 14: Urutan DFS dari Node 0 (angka dalam kurung = urutan kunjungan)

6.5 Post Test: BFS dan DFS pada Graf Pre Test (Gambar 6.4)

Source Code Python

```
from collections import deque

def bfs(graph, start, n):
    visited = [False] * n
    urutan = []
    queue = deque()

    # tandai simpul awal dan masukkan ke antrian
    visited[start] = True
    queue.append(start)

    while queue:
        node = queue.popleft()
        urutan.append(node)

        # kunjungi tetangga yang belum dikunjungi
        for tetangga in graph[node]:
```

```

        if not visited[tetangga]:
            visited[tetangga] = True
            queue.append(tetangga)

    # tangani simpul tidak terjangkau
    if not queue:
        for j in range(n):
            if not visited[j]:
                visited[j] = True
                queue.append(j)
                break

    return urutan

def _dfs_rekursif(graph, node, visited, urutan):
    # tandai dan catat simpul yang dikunjungi
    visited[node] = True
    urutan.append(node)

    # rekursif ke tetangga yang belum dikunjungi
    for tetangga in graph[node]:
        if not visited[tetangga]:
            _dfs_rekursif(graph, tetangga, visited, urutan)

def dfs(graph, start, n):
    visited = [False] * n
    urutan = []

    _dfs_rekursif(graph, start, visited, urutan)

    # tangani simpul yang belum terjangkau
    for j in range(n):
        if not visited[j]:
            _dfs_rekursif(graph, j, visited, urutan)

    return urutan

```

Jawaban 1: Penerapan BFS dan DFS pada graf Gambar 6.4

Program dijalankan pada graf pre test (Gambar 6.4) dengan 7 simpul mulai dari node 0:

Tabel 15: Validasi Hasil Program vs Manual pada Graf Pre Test

Algoritma	Hasil Program	Jawaban Manual Pre Test	Sama?
BFS	[0, 1, 2, 3, 4, 5, 6]	[0, 1, 2, 3, 4, 5, 6]	Ya
DFS	[0, 1, 3, 5, 2, 4, 6]	[0, 1, 3, 5, 2, 4, 6]	Ya

Jawaban 2: Analisis kesesuaian hasil program dengan jawaban pre test

Hasil program sepenuhnya sesuai dengan jawaban manual pre test untuk kedua algoritma. Ini memvalidasi bahwa:

1. Implementasi Python BFS menggunakan deque sudah benar dalam mengeksplorasi graf secara berlapis, menghasilkan urutan yang identik dengan penelusuran manual menggunakan antrian.
2. Implementasi Python DFS rekursif sudah benar dalam mengeksplorasi graf secara mendalam, menghasilkan urutan yang identik dengan penelusuran manual menggunakan prinsip backtracking.
3. Penanganan simpul tidak terjangkau (isolated nodes) sudah ditangani dengan benar di kedua implementasi sehingga seluruh 7 simpul berhasil dikunjungi.

6.6 Implementasi Nyata

Algoritma BFS dan DFS merupakan fondasi dari banyak aplikasi dunia nyata mulai dari navigasi, jaringan sosial, hingga analisis jaringan komputer. Jurnal berikut mendukung relevansinya.

[1] Penulis: Ahmed, R., Chekol, B., dan Alsaleh, M. (2025)

Judul: "HDBMS: A Context-Aware Hybrid Graph Traversal Algorithm for Efficient Information Discovery in Social Networks"

Publikasi: arXiv preprint arXiv:2508.14092, Uskudar University, Istanbul, 2025.

Intisari: Makalah ini mengusulkan algoritma traversal hibrid berbasis BFS dan DFS yang dinamis dan adaptif untuk jaringan sosial. Eksperimen pada berbagai graf berarah menunjukkan bahwa BFS unggul dalam menemukan jalur terpendek antar pengguna (misal pencarian teman terdekat), sedangkan DFS lebih efisien dalam menelusuri rantai hubungan yang dalam (misal rantai influencer). Karya ini menegaskan bahwa pemilihan algoritma traversal harus disesuaikan dengan konteks dan struktur graf yang dihadapi.

Link: <https://arxiv.org/pdf/2508.14092>

Kelebihan dalam konteks nyata:

- BFS unggul untuk pencarian jalur terpendek pada jaringan tidak berbobot seperti jaringan sosial (degree of separation), routing paket jaringan, dan rekomendasi koneksi.
- DFS unggul untuk eksplorasi mendalam seperti deteksi siklus dalam dependency graph, topological sort pada build system, dan traversal struktur file sistem operasi.

- Kompleksitas $O(V+E)$ yang sama membuat keduanya efisien untuk graf skala menengah dan dapat dioptimalkan lebih lanjut dengan teknik paralel untuk graf berskala besar.

Kekurangan dalam konteks nyata:

- BFS membutuhkan memori besar $O(V)$ untuk antrian, tidak cocok untuk graf dengan jutaan simpul seperti graf internet tanpa optimasi memori.
- DFS berisiko stack overflow pada graf sangat dalam apabila diimplementasikan secara rekursif, dan perlu diubah ke iteratif berbasis stack eksplisit untuk sistem produksi.

DAFTAR PUSTAKA

- [1] R. Ahmed and S. I. Aug, “HDBMS : A Context-Aware Hybrid Graph Traversal Algorithm for Efficient Information Discovery in Social Networks,” pp. 1–19, arXiv: arXiv:2508.14092v1.
- [2] S. Ketchaya and A. Rattanatanurak, “Parallel Multi - Deque Partition Dual - Deque Merge sorting algorithm using OpenMP,” *Scientific Reports*, no. 0123456789, pp. 1–13, 2023, ISSN: 2045-2322. DOI: 10.1038/s41598-023-33583-4
- [3] P. Ganapathi and R. Chowdhury, “Parallel Divide-and-Conquer Algorithms for Bubble Sort , Selection Sort and Insertion Sort,” vol. 65, no. 10, 2022.
- [4] I. Ganti and E. Hutt, “Comparing and Reviewing Modern Primality Tests Significance of Primes and Primality Tests,” vol. 11, no. 3, pp. 1–11, 2022.
- [5] S. Sundaramoorthy and G. Karunanidhi, “A systematic analysis on performance and computational complexity of sorting algorithms,” 2025.
- [6] X. Wei, C. Wu, H. Yu, S. Liu, and Y. Yuan, “A coin selection strategy based on the greedy and genetic algorithm,” *Complex & Intelligent Systems*, vol. 9, no. 1, pp. 421–434, 2023, ISSN: 2198-6053. DOI: 10.1007/s40747-022-00799-2
- [7] E. Dika, “Results of comparison between different sorting algorithms,” vol. 2, pp. 39–52, 2021.